

Autonomous Workspace Architecture

A Comprehensive Plan for a Private, Multi-Agent Notion Clone

1. Executive Summary

This document defines the architecture for a "Notion Clone" enhanced by a native, localized AI agentic system. Unlike standard AI integrations, this system is built to be **fully private, offline-first, and contextually aware** of the entire user workspace without excessive RAM consumption.

2. Technical Stack & Core Engines

Frontend	TypeScript / React (Tauri Shell)
System Layer	Rust (Tauri IPC & Sub-process management)
AI Backend	Python (FastAPI, LangGraph)
Inference	llama-cpp-python (GGUF: Llama-3.2 / Phi-3)
Vector Memory	ChromaDB (Local persistent storage)
State Memory	SQLite (Agentic history and plugin registry)

3. The Dual-Database Communication Bridge

To ensure application stability and performance, the system utilizes an **Event-Driven Synchronous Bridge**.

Architecture Rule: The Main App DB (Rust) and the AI Brain DB (Python) never touch the same files. They communicate via an internal `localhost` HTTP interface managed by Rust.

- **The Save Flow:** When a user stops typing, TypeScript triggers a save to Rust. Rust saves to the SQLite main DB and immediately sends a background POST request to the Python `/sync_page` endpoint.
- **Semantic Indexing:** Python receives the text, chunks it using recursive character splitting, and generates embeddings. It updates only the specific *Collection* related to that page ID in ChromaDB.

4. Multi-Agent Reasoning Pipeline (LangGraph)

The AI does not generate text in one pass. It uses a three-stage "Thinking Loop" to ensure accuracy and persona-consistency.

Node 1: The Researcher

Accesses the Vector DB. It pulls relevant context from the current page and any uploaded PDFs. It filters for facts only.

Node 2: The Writer (Persona Engine)

Receives the facts. It applies the user-selected **Tone** (Academic, Business, Friendly, Lecturer) to draft the response. It follows strict Markdown formatting rules.

Node 3: The Reviewer (Anti-Hallucination)

Acts as a quality gate. It compares the Writer's draft against the Researcher's facts. If the Writer added "hallucinated" info, the Reviewer sends the draft back for a rewrite.

5. Agentic "Hands" & Tool Calling

The AI is capable of modifying the application UI via JSON Command Emission. Tools are registered dynamically based on active plugins.

```
{
  "agent_action": "modify_ui",
  "tool": "insert_todo_list",
  "parameters": {
    "items": ["Draft budget", "Review Q1 goals"],
    "location": "current_cursor"
  }
}
```

This payload is sent from Python to Rust, which then emits a Tauri event to the TypeScript frontend to render the new block instantly.

6. RAM Optimization Strategy

Running local LLMs requires strict memory management:

- **Quantization:** Use 4-bit (Q4_K_M) GGUF models to reduce VRAM/RAM footprint by ~70%.
- **Context Windowing:** Only the current active page and recently interacted PDFs are loaded into the Vector DB's active memory.

- **Lazy Loading:** The Python server remains in a "warm" idle state, only activating the LLM weights when a generation request is sent.

7. Development Roadmap

Phase 1: The Sidecar (Week 1-2)

Set up Rust to spawn the Python process. Establish the internal HTTP handshake and basic "Ask" functionality.

Phase 2: Semantic Memory (Week 3-4)

Implement ChromaDB and the background sync service. Ensure the AI can "remember" content from different pages.

Phase 3: Agentic Reasoning (Week 5-6)

Build the LangGraph Researcher/Writer/Reviewer loop. Implement the "Persona" selection UI.

Phase 4: Tool Integration (Week 7+)

Enable "Hands" (Tool calling) so the AI can create blocks, calendar events, and budget entries.